UNIVERSITAS INDONESIA
Veritas, Probitas, Iustitia

FAKULTAS ILMU KOMPUTER

CSCE604029 • Computer Graphics
Semester Gasal 2026/2027
Fakultas Ilmu Komputer, Universitas Indonesia

Assignment 2: Halfedge Mesh Processing

Release: 18 October 2026 **Deadline: 30 October 2026, 23.55 WIB**

Alignment with CPMK

- Menguasai konsep-konsep dasar grafika komputer secara teoretis seperti model kamera, properti warna dan cahaya, rendering, iluminasi, dan geometri.
- Dapat mendesain solusi grafika komputer untuk pembentukan citra digital menggunakan teknik pemodelan dasar, terutama menggunakan metode berbasis rasterization dan ray tracing.

1 Introduction

This assignment covers the **halfedge mesh** data structure and three operations built on it: **edge flip**, **linear subdivision**, and **Catmull-Clark subdivision**. A halfedge mesh stores connectivity explicitly, so adjacency queries around a vertex, a face, or an edge follow pointers instead of searching every face. You will implement the traversal and geometry queries first, then a local operation that changes connectivity in place, and finally two global operations that build a refined mesh.

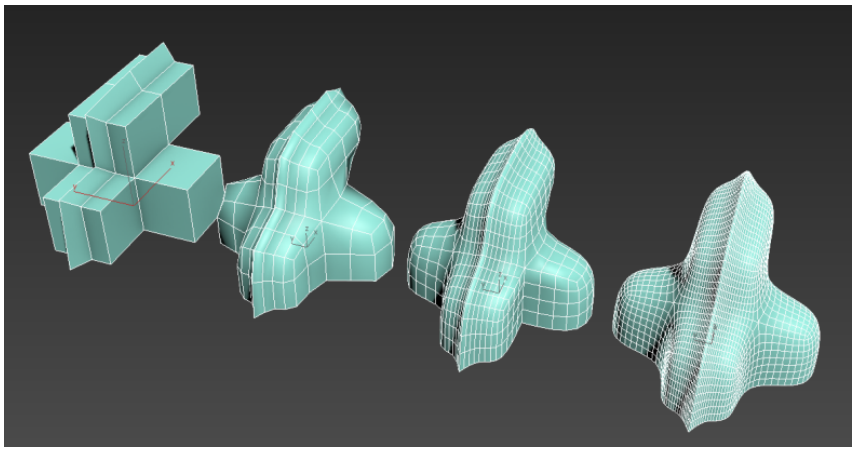


Figure 1: Repeated subdivision of a coarse control mesh.

2 Supporting Material

The concepts in this assignment are covered in the Geometry lecture (Modul 5, *Geometric Representation*). Page numbers below refer to the PDF of the slides. Concepts that the slides show only briefly are explained in more detail in this section.

2.1 Halfedge Mesh

A halfedge mesh splits every edge into two oppositely directed *halfedges*. Each halfedge belongs to exactly one face, the face on its left. The slides introduce the elements on pages 50 to 58.

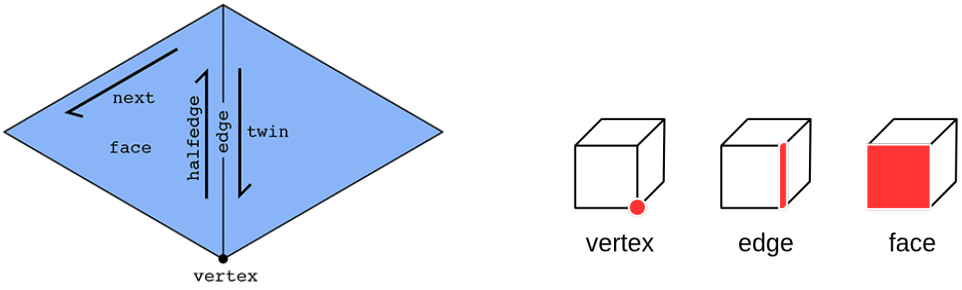


Figure 2: A halfedge with its twin, next, edge, face, and vertex (left), and the three mesh elements it connects (right).

The starter code in `halfedge.py` uses the following conventions. Read them before starting, because every task depends on them.

- `h.vertex` is the **target** vertex of halfedge `h`, as on slide page 54. The source vertex is `h.twin.vertex`.
- `h.next` is the next halfedge around the same face, in counter-clockwise order when the face is seen from outside the surface.
- `v.halfedge` is an **outgoing** halfedge of vertex `v`. A vertex that belongs to no face has `v.halfedge = None`.
- `f.halfedge` is one halfedge of face `f`, and `e.halfedge` is one of the two halfedges of edge `e`.
- Every element has an integer `index` equal to its position in `mesh.vertices`, `mesh.edges`, `mesh.faces`, or `mesh.halfedges`.

Boundary loops

An open surface, such as `grid.obj`, has edges that belong to only one face. The starter represents each hole in the surface by one **boundary face**: a face object with `is_boundary = True`, stored in `mesh.boundary_faces` and not in `mesh.faces`. The halfedges on the hole side of the boundary edges belong to that boundary face, and their `next` pointers form a closed loop around the hole. As a result, every halfedge has a twin, a next halfedge, and a face, and both traversal patterns below terminate on open meshes as well. For a vertex on the boundary, `v.halfedge` is the outgoing halfedge whose twin belongs to a boundary face.

2.2 Traversal

Slide page 59 lists the two traversal patterns used in this assignment.

- Walking around a face follows `h = h.next` until the walk returns to the starting halfedge.
- Circulating around a vertex follows `h = h.twin.next` until the circulation returns to the starting halfedge. With the conventions above, every halfedge visited is an outgoing halfedge of the vertex.

Slide pages 61 and 62 apply these patterns to compute a face centroid and the valence of a vertex. Each traversal touches only the elements around one face or one vertex, so its cost depends on the number of sides or the valence, not on the size of the mesh.

2.3 Face Geometry

Slide page 63 gives the normal of a polygon with vertices $v_0, \ldots, v_{n-1}$ by Newell's method,

$$\mathbf{n} = \sum_{i=0}^{n-1} (v_i - v_{i+1}) \times (v_i + v_{i+1}),$$

with indices taken modulo n, and the area of a polygon by fan triangulation from v_0,

$$A = \frac{1}{2} \sum_{i=1}^{n-2} \left\| (v_i - v_0) \times (v_{i+1} - v_0) \right\|.$$

The centroid of a face is the average of its vertex positions (slide page 60).

2.4 Euler Characteristic

For a mesh with V vertices, E edges, and F interior faces, the *Euler characteristic* is $\chi = V - E + F$. The value depends on the topology of the surface and not on how finely it is meshed. A closed surface without holes, such as `cube.obj`, has $\chi = 2$. A closed surface with one handle, such as `torus.obj`, has $\chi = 0$. An open surface shaped like a disc, such as `grid.obj`, has $\chi = 1$. Boundary faces are not counted in F.

2.5 Edge Flip

Slide page 64 shows several local operations that edit a mesh in place. This assignment uses the *edge flip*. Consider an interior edge between vertices a and b whose two adjacent faces are the triangles (a, b, c) and (b, a, d). Together they form a quadrilateral a, d, b, c. Flipping the edge replaces the diagonal ab of that quadrilateral with the other diagonal cd, so the two triangles become (c, a, d) and (d, b, c), up to a rotation of the vertex order.

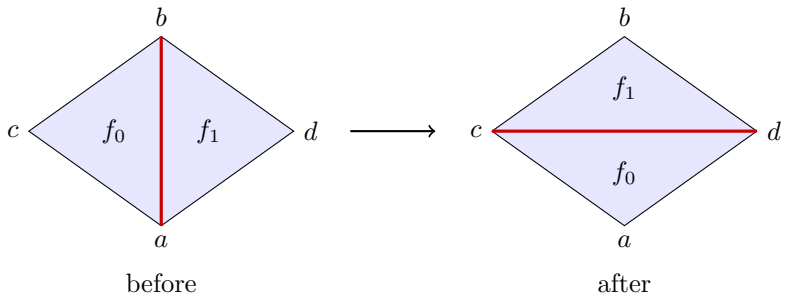


Figure 3: Flipping the edge ab shared by face $f_0 = (a, b, c)$ and face $f_1 = (b, a, d)$.

An edge flip creates and deletes no element. The same two faces, the same edge, and the same six halfedges are reused, and only their pointers change. The number of vertices, edges, and faces stays the same. Figure 4 shows a flip on `fan.obj`.

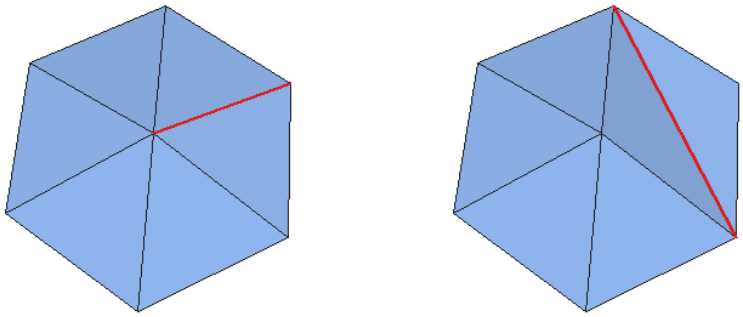


Figure 4: Edge 0 of `fan.obj` (red) before and after the flip, produced by `python run.py meshes/fan.obj --flip 0 --elevation 75 --png fan-flip.png` with the answer key.

2.6 Subdivision

Subdivision increases the resolution of a mesh (slide pages 67 and 71). Both schemes in this assignment insert the same new points and produce the same connectivity. They differ only in the positions of the points.

Linear subdivision

Linear subdivision (slide page 72) inserts one point at the midpoint of every edge and one point at the centroid of every face, then reconnects them. An n-sided face becomes n quadrilaterals. Each quadrilateral contains one corner v_i of the original face, the midpoint of the edge from v_i to the next corner, the face centroid, and the midpoint of the edge from the previous corner to v_i. Original vertices keep their positions.

Catmull-Clark subdivision

Catmull-Clark subdivision (slide pages 74 to 81) uses the same connectivity but moves the points to make the surface smoother.

- The **face point** of a face is its centroid (slide page 77).
- The **edge point** of an edge with endpoints v_0, v_1 and adjacent face points f_0, f_1 is $E = (v_0 + v_1 + f_0 + f_1)/4$ (slide page 78).
- The new position of an original vertex v is

$$v' = \frac{Q + 2R + (n-3)S}{n},$$

  where n is the number of faces adjacent to v, Q is the average of the face points of those faces, R is the average of the midpoints of the n edges incident to v, and S is the original position of v (slide pages 79 and 80).

The edge point rule needs two adjacent faces, so this assignment applies Catmull-Clark subdivision to closed meshes only.

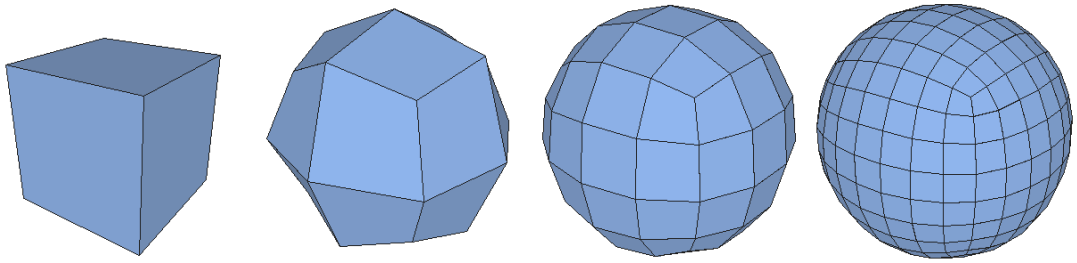


Figure 5: `cube.obj` and the results of one, two, and three Catmull-Clark steps, rendered by `run.py` with the answer key.

3 Starter Package

3.1 Files

The starter package on SCeLE contains the following files. Implement every task in `student.py`. The other Python files are used as provided; the grader replaces them with its own copies.

File	Content and role	Modify?
`student.py`	The functions of Tasks 1 to 6. Every function raises `NotImplementedError` until you implement it. This is the only Python file you submit.	Yes
`halfedge.py`	Classes `Vertex`, `Edge`, `Face`, `Halfedge`, and `HalfedgeMesh`. `HalfedgeMesh.from_polygons(positions, polygons)` builds a mesh, including boundary loops, from vertex positions and polygons given as lists of vertex indices. `mesh.validate()` checks the pointer invariants and raises `MeshError` on the first violation.	No
`support.py`	Reading and writing OBJ files and rendering a mesh to PNG. Writing and rendering call your `face_vertices`; shading uses your `face_normal` once it is implemented.	No
`run.py`	Command-line runner that loads a mesh, applies your operations, validates the result after every operation, and prints, writes, or renders it.	No
`meshes/`	Eight OBJ files: `cube`, `pyramid`, `lshape`, `torus`, `octahedron`, and `tetrahedron` are closed; `grid` and `fan` are open.	Add your own
`requirements.txt`	Dependencies: NumPy and Pillow.	No

3.2 Running the Code

From the `starter/` directory, install the dependencies and check that every provided mesh loads:

```
python -m pip install -r requirements.txt
python run.py meshes/cube.obj --check-input
```

After implementing the tasks, the following commands print the summary of Task 3, flip an edge, and apply subdivision:

```
python run.py meshes/cube.obj --summary
python run.py meshes/fan.obj --flip 0 --png fan-flip.png
python run.py meshes/cube.obj --subdivide linear --obj cube-linear.obj
python run.py meshes/cube.obj --subdivide catmull-clark --levels 3 --png
cc3.png
```

Option `--flip` accepts several edge indices and flips them in the given order. Options `--azimuth` and `--elevation` rotate the camera of the rendered PNG, in degrees. Until the functions a command needs are implemented, the runner prints the first unfinished task and writes no file. If one of your functions raises `ValueError`, the runner prints `Operation rejected` with your message.

On the teaching team's laptop (Python 3.12, NumPy 2.5), the answer key runs every task on all eight provided meshes in less than one second. Five Catmull-Clark steps on `cube.obj`, which produce 6,144 faces, take about 0.15 seconds in total (median of seven runs).

4 Tasks

Tasks 1 to 6 are worth 85 points and the report in Section 5 is worth 15 points. The functions are graded with hidden meshes in addition to the provided ones, so test your code on meshes of your own as well.

Task 1: Traversal (12 points)

Implement the following functions with the traversal patterns of slide page 59.

- `face_halfedges(face)` returns the list of halfedges of the face, starting with `face.halfedge` and following the `next` pointers.
- `face_vertices(face)` returns the target vertices of the halfedges from `face_halfedges(face)`, in the same order. For a face built from the OBJ line `f 3 1 4`, the result is the vertices with indices 2, 0, and 3, in that order.
- `vertex_halfedges(vertex)` returns the list of outgoing halfedges of the vertex, starting with `vertex.halfedge` and following `h = h.twin.next`.
- `vertex_neighbors(vertex)` returns the vertices connected to the vertex by an edge, in the order of `vertex_halfedges(vertex)`.
- `vertex_faces(vertex)` returns the interior faces that contain the vertex, in the order of `vertex_halfedges(vertex)`. Boundary faces are not included.
- `is_boundary_vertex(vertex)` returns `True` if the vertex lies on a boundary loop and `False` otherwise.

Each list contains every element once. The vertex functions return an empty list, or `False`, for a vertex with `halfedge = None`. The functions of this task must not iterate over `mesh.faces`, `mesh.halfedges`, or any other list of the whole mesh.

Task 2: Face Geometry (12 points)

- `face_centroid(face)` returns the average of the vertex positions as a NumPy *array* with *shape* $(3,)$.
- `face_normal(face)` returns the unit normal computed with Newell's method as an *array* with *shape* $(3,)$. The normal points to the side from which the face vertices appear counter-clockwise. If the length of the Newell vector is below 10^{-12}, return the zero vector.

- `face_area(face)` returns the area of the face as a Python `float`.

Every face is planar and does not intersect itself, but a face may be non-convex, as the two caps of `lshape.obj` are. The area of the cap of an L-shaped prism is the area of the L shape. The *return value* of each function must not depend on which halfedge of the face `face.halfedge` refers to.

Task 3: Mesh Summary (6 points)

`mesh_summary(mesh)` returns a `dict` with exactly the following keys.

Key	Value
`vertices`, `edges`, `faces`, `halfedges`	Number of elements in the corresponding list of the mesh. `faces` counts interior faces only; `halfedges` includes the halfedges of boundary faces.
`triangles`, `quads`, `polygons`	Number of interior faces with three sides, four sides, and more than four sides.
`boundary_loops`	Number of boundary faces.
`boundary_vertices`	Number of vertices for which `is_boundary_vertex` is `True`.
`euler_characteristic`	$V - E + F$ with F the number of interior faces.
`surface_area`	Sum of `face_area` over all interior faces, as a `float`.

All other values are Python `int`.

Task 4: Edge Flip (20 points)

`flip_edge(mesh, edge)` flips `edge` in place and returns the same `Edge` object. Use the names of the supporting material: before the flip the edge joins a and b, and its faces are the triangles (a, b, c) and (b, a, d).

- After the flip, the edge joins c and d, its two faces contain the vertices $\{c, a, d\}$ and $\{d, b, c\}$, and `mesh.validate()` succeeds.
- The function reuses the existing elements. It creates no vertex, edge, face, or halfedge, removes none, and does not change any `index`.
- Every `v.halfedge` and `f.halfedge` still satisfies the conventions in the supporting material, including the convention for boundary vertices.
- The function raises `ValueError` with a short message and leaves the mesh unchanged if the edge lies on a boundary, if one of its two faces is not a triangle, or if c and d are already joined by an edge.

Flipping the same edge twice must return the mesh to its original connectivity.

Task 5: Linear Subdivision (15 points)

`subdivide_linear(mesh)` returns a new `HalfedgeMesh` after one step of linear subdivision and leaves the input mesh unchanged. It must work on open and closed meshes. Build the result with `HalfedgeMesh.from_polygons` so that the indices follow this order, where V and E are the numbers of vertices and edges of the input mesh:

- vertex i of the result, for $i < V$, is input vertex i;

- vertex $V + j$ is the point inserted on input edge j;
- vertex $V + E + k$ is the point inserted in input face k.

The polygons are listed face by face in the order of `mesh.faces`. Within a face, they follow the order of `face_halfedges(face)`. For halfedge `h` the polygon is

[`h.vertex`, point of `h.next.edge`, point of the face, point of `h.edge`].

Task 6: Catmull-Clark Subdivision (20 points)

`subdivide_catmull_clark(mesh)` returns a new `HalfedgeMesh` after one step of Catmull-Clark subdivision with the rules of the supporting material and leaves the input mesh unchanged. The vertex order and the polygons are the same as in Task 5; only the positions differ. The function raises `ValueError` if the mesh has a boundary loop. A vertex with `halfedge = None` keeps its position.

5 Testing and Report

5.1 Testing

Validate your functions with numerical checks in addition to the rendered images. Include checks that exercise the following behavior, using the provided meshes and meshes of your own in OBJ format:

- Traversal and geometry on every provided mesh, including the open meshes and the caps of `lshape.obj`.
- The same face stored with a different starting halfedge.
- Edge flips that should be accepted and each kind of edge that should be rejected.
- The numbers of vertices, edges, and faces after one and several steps of both subdivision schemes, on triangles, quadrilaterals, and faces with more than four sides.

5.2 Report

Answer the following questions in a PDF report of at most four pages. Support every claim with numbers or images produced by your code, and put the command that produced each result next to it.

1. (5 points) Describe the checks you wrote for Tasks 1 to 6 and their results. Include a table with `mesh_summary` of every provided mesh before and after one step of each subdivision scheme that applies to it. Compare V, E, F, and χ before and after subdivision, and explain the relations you observe from the way the scheme inserts points and faces.
2. (5 points) Try to flip every edge of `octahedron.obj`, `fan.obj`, `pyramid.obj`, and `tetrahedron.obj`. Report how many edges are accepted and how many are rejected for each reason. For each rejection condition in Task 4, explain what would be wrong with the mesh if the flip were carried out anyway.
3. (5 points) Apply one to four steps of linear subdivision and of Catmull-Clark subdivision to `cube.obj`. Report the number of faces, the surface area, and the number of vertices whose valence is not 4 after every step. Compare the two schemes and explain the differences you observe with their rules.

Technical Constraints

- Do not use mesh processing libraries such as trimesh, Open3D, PyMeshLab, or libigl, because the assignment assesses your own implementation of halfedge traversal and subdivision. NumPy and the Python standard library may be used.
- Do not modify `halfedge.py`, `support.py`, or `run.py`, and do not change the function signatures in `student.py`. The grader imports your `student.py` together with its own copies of the other files.
- Do not add attributes to the mesh classes or store results in global variables between calls, because the grader calls each function on meshes it builds itself.

6 Submission Format

This assignment is done **individually**. Submit one ZIP archive on SCeLE named `<NPM>-A2.zip`, containing:

- `student.py` with your implementation of Tasks 1 to 6;
- `report.pdf` with the answers to the questions in Section 5;
- a folder `meshes/` with the OBJ files you created for testing, if any.

Submit through SCeLE before the deadline, 30 October 2026 at 23.55 WIB.

6.1 Post-submission Quiz

After the submission, each student takes a programming quiz. The quiz extends the halfedge mesh code of this assignment with short functions. It verifies individual understanding of the submitted implementation. Assignment grading depends on the quiz result, as stated in the course plan; the scoring rule and the quiz schedule will be announced by the teaching team.

Important Notes

- **Originality.** The submitted code and report must be your own work. Code from other students, including students of previous years, is not allowed.
- **Importable code.** `student.py` must be importable without errors and must not print, read, or write files when it is imported.
- **Reproducibility.** Every number and image in the report must be reproducible with the submitted `student.py`, the provided starter, and the command written next to it.

7 Resources

- Course materials on geometric representation (Modul 5), in particular the halfedge data structure, local operations, and subdivision.
- Marschner et al., *Fundamentals of Computer Graphics*, 4th edition, recommended in the course plan.
- E. Catmull and J. Clark, *Recursively generated B-spline surfaces on arbitrary topological meshes*, Computer-Aided Design 10(6), 1978.

- Jerry Yin, *Half-edge data structure*, interactive demo referenced on slide page 58: `https://jerryyin.info/geometry-processing-algorithms/half-edge/`.

8 Contact

Lecturer: Ikhsanul Habibie (`ikhsanul.habibie12@cs.ui.ac.id`).
Teaching assistants: Syahrul Apriansyah and Alpha Sutha Media.

Happy Meshing!